

Características de Qualidade de Repositórios Java no GitHub

Lab02 — Experimentação de Software

Kaio Henrique Oliveira da Silveira Barbosa

1. Introdução

O desenvolvimento colaborativo de software open-source apresenta desafios inerentes à manutenção da qualidade interna do código. À medida que um projeto cresce em popularidade e recebe contribuições de múltiplos desenvolvedores, atributos como modularidade, coesão e manutenibilidade tornam-se difíceis de preservar. Práticas como revisão de código e análise estática via ferramentas de CI/CD buscam mitigar esses riscos.

Este trabalho analisa empiricamente a qualidade interna de repositórios Java hospedados no GitHub, correlacionando métricas de produto — calculadas pela ferramenta CK — com características do processo de desenvolvimento de cada projeto. O estudo parte da hipótese geral de que projetos mais populares, maduros e ativos tendem a apresentar melhores indicadores de qualidade interna, dado o maior escrutínio da comunidade sobre o código.

A análise foi conduzida sobre uma amostra dos top-1.000 repositórios Java por número de estrelas, coletados via API GraphQL do GitHub. Após o processamento, 978 repositórios foram analisados com sucesso — os demais foram excluídos por limitações técnicas.

2. Questões de Pesquisa e Hipóteses

Para orientar a investigação, foram formuladas quatro questões de pesquisa, cada uma relacionando uma característica do processo de desenvolvimento com as métricas de qualidade obtidas pelo CK.

RQ	Variável de Processo	Hipótese Informal	Resultado
RQ01	Popularidade (estrelas)	Projetos mais populares terão menor CBO e maior coesão (LCOM), dado maior volume de revisões externas.	Não confirmada
RQ02	Maturidade (idade em anos)	Projetos mais antigos possuem arquitetura mais consolidada, mas podem acumular dívida técnica — DIT ligeiramente maior e LCOM variável.	Parcialmente confirmada
RQ03	Atividade (nº de releases)	Projetos com ciclos de release frequentes indicam manutenção ativa, favorecendo melhores CBO e LCOM.	Não confirmada
RQ04	Tamanho (LOC + comentários)	Projetos maiores tendem a ser mais complexos, com CBO e DIT mais altos; muitos comentários podem indicar código legado.	Confirmada

Tabela 1 — Questões de pesquisa, variáveis de processo e hipóteses informais.

3. Metodologia

O experimento foi conduzido inteiramente por meio de scripts Python. A ferramenta CK (versão 0.7.0) foi responsável pela análise estática do código-fonte Java, gerando arquivos .csv com as métricas por classe.

Ferramenta / Recurso	Finalidade
Python 3.x	Scripts de coleta, processamento e análise
GitHub GraphQL API	Coleta dos 1.000 repositórios Java
CK 0.7.0	Análise estática de métricas de qualidade Java
Git (--depth=1)	Clonagem superficial dos repositórios
matplotlib	Geração de gráficos de dispersão
scipy	Cálculo da correlação de Spearman
python-dotenv	Gerenciamento do token de acesso ao GitHub

Tabela 2 — Ferramentas e recursos utilizados.

O estudo emprega dois conjuntos de métricas: métricas de processo, que caracterizam o repositório em termos de desenvolvimento, e métricas de qualidade, calculadas pela análise estática do código-fonte.

Sigla	Nome Completo	Interpretação
CBO	Coupling Between Objects	Acoplamento entre classes — valores altos indicam maior dependência entre módulos.
DIT	Depth Inheritance Tree	Profundidade da herança — valores altos indicam hierarquias complexas.
LCOM	Lack of Cohesion of Methods	Falta de coesão — valores altos indicam classes com responsabilidades difusas.

Tabela 3 — Métricas de qualidade calculadas pela ferramenta CK.

Cada métrica de qualidade é sumarizada por repositório com mediana, média e desvio padrão. A mediana é adotada como medida principal por ser robusta à presença de outliers.

3.1 Fluxo de Execução

O processo de coleta e análise foi estruturado em quatro etapas sequenciais. A etapa de coleta utiliza a API GraphQL do GitHub para obter os repositórios mais populares em Java. Em seguida, cada repositório é clonado superficialmente para execução do CK. Os resultados individuais são consolidados em um único arquivo e, por fim, a análise estatística gera os gráficos de correlação e os valores de Spearman.

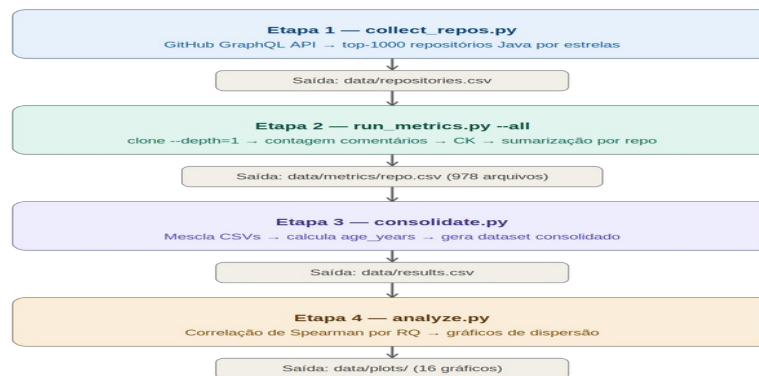


Figura 1 — Fluxo de execução do experimento: coleta → métricas → consolidação → análise.

3.2 Desafios e Tomadas de Decisão

Desafio 1 — Ausência de linhas de comentário no CK: O CK 0.7.0 não expõe linhas de comentário. Decisão: implementar a contagem diretamente no script `run_metrics.py` via expressões regulares.

Desafio 2 — Incompatibilidade com Java moderno: Repositórios com sintaxe Java 14+ causam crash no CK. Decisão: excluir os 22 repositórios afetados. A amostra de 978 permanece representativa.

Desafio 3 — Falha na remoção de clones no macOS: Repositórios grandes bloqueavam `shutil.rmtree()`. Decisão: tratar como erro não-crítico — as métricas já foram coletadas com sucesso.

Desafio 4 — Erro 502 na API GraphQL: A query com `first: 100` gerava erro 502. Decisão: reduzir para `first: 50`, eliminando o problema.

4. RQ01 — Popularidade vs. Qualidade

A popularidade dos repositórios é mensurada pelo número de estrelas no GitHub. A hipótese formulada era de que projetos com maior visibilidade tendem a receber mais revisões externas, o que favoreceria uma arquitetura mais cuidadosa — traduzindo-se em menor CBO e menor LCOM.

Medida	Valor
Mediana	5.793
Média	9.637,95
Desvio Padrão	11.782,20

Tabela 4 — Estatísticas descritivas: número de estrelas ($n = 978$).

A alta assimetria evidenciada pelo desvio padrão muito superior à mediana indica que poucos repositórios concentram a maior parte das estrelas. Nenhuma das três correlações foi estatisticamente significativa ($p > 0,05$ em todos os casos), com valores de p próximos de zero.

Métrica	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,030	0,357	Não
DIT (mediana)	-0,039	0,223	Não
LCOM (mediana)	0,008	0,813	Não

Tabela 5 — Correlação de Spearman: estrelas $\times$ métricas de qualidade.

Os gráficos abaixo evidenciam a ausência de qualquer padrão entre o número de estrelas e as métricas de qualidade. Os pontos estão dispersos sem tendência visual clara, corroborando os valores de p próximos de zero. A hipótese inicial é portanto refutada: popularidade no GitHub mede visibilidade e adoção, não arquitetura interna.

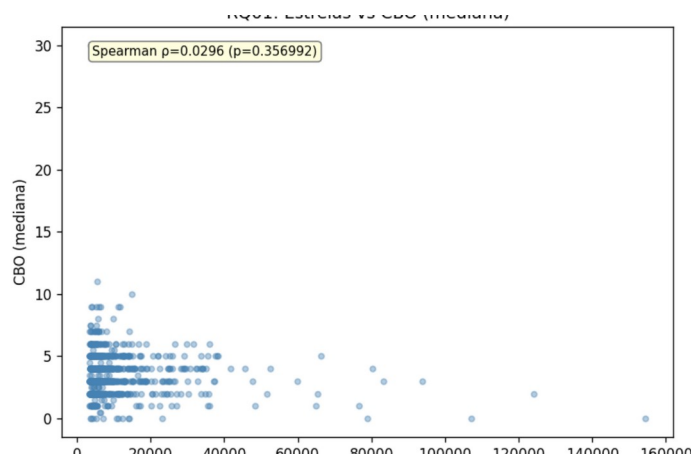


Figura 2 — Dispersão: estrelas $\times$ CBO. Spearman $\rho = 0,030$ ($p = 0,357$). Nuvem plana, sem tendência.

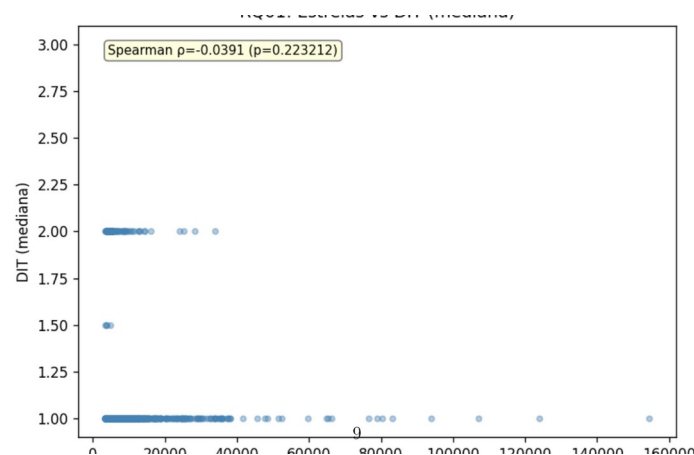


Figura 3 — Dispersão: estrelas $\times$ DIT. Spearman $\rho = -0,039$ ($p = 0,223$). Ausência de padrão.

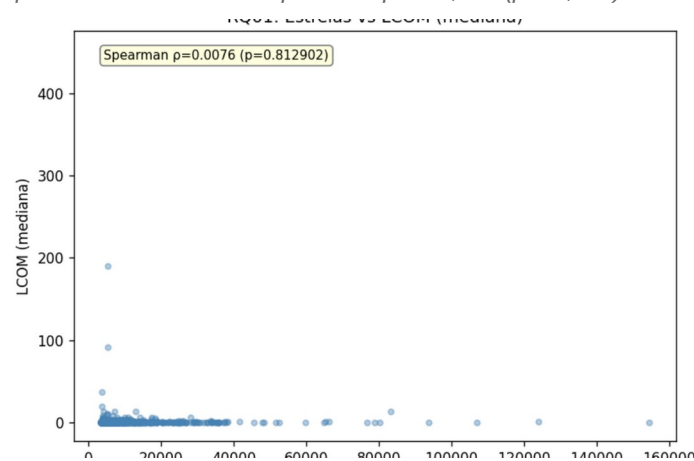


Figura 4 — Dispersão: estrelas $\times$ LCOM. Spearman $\rho = 0,008$ ($p = 0,813$). Nenhuma correlação.

Nota: Hipótese refutada. ρ próximo de zero nas 3 métricas, nenhum p-value significativo. Estrela mede visibilidade — repositórios de tutoriais e frameworks populares acumulam estrelas sem necessariamente ter boa arquitetura interna.

5. RQ02 — Maturidade vs. Qualidade

A maturidade de um repositório é representada pela sua idade em anos. A hipótese era de que projetos mais antigos apresentariam DIT ligeiramente mais elevado — fruto da estratificação natural das hierarquias ao longo do tempo — e LCOM variável, decorrente de refatorações incompletas.

Medida	Valor
Mediana	10,26 anos
Média	10,11 anos
Desvio Padrão	3,16 anos

Tabela 6 — Estatísticas descritivas: idade dos repositórios em anos ($n = 978$).

A distribuição da idade é relativamente simétrica (mediana $\approx$ média), com baixa dispersão. A correlação entre maturidade e CBO não foi significativa. No entanto, DIT ($\rho = 0,098$, $p = 0,002$) e LCOM ($\rho = 0,091$, $p = 0,005$) apresentaram correlações fracas mas significativas: projetos mais antigos tendem a ter hierarquias ligeiramente mais profundas e classes levemente menos coesas.

Métrica	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,020	0,541	Não
DIT (mediana)	0,098	0,002	Sim
LCOM (mediana)	0,091	0,005	Sim

Tabela 7 — Correlação de Spearman: idade (anos) $\times$ métricas de qualidade.

Os gráficos abaixo mostram que os dados de idade se distribuem de forma homogênea entre 5 e 15 anos. Para CBO, a nuvem de pontos não apresenta inclinação visível. Para DIT e LCOM, uma leve tendência positiva pode ser percebida nos repositórios com mais de 10 anos, embora a magnitude seja pequena. A hipótese é parcialmente confirmada.

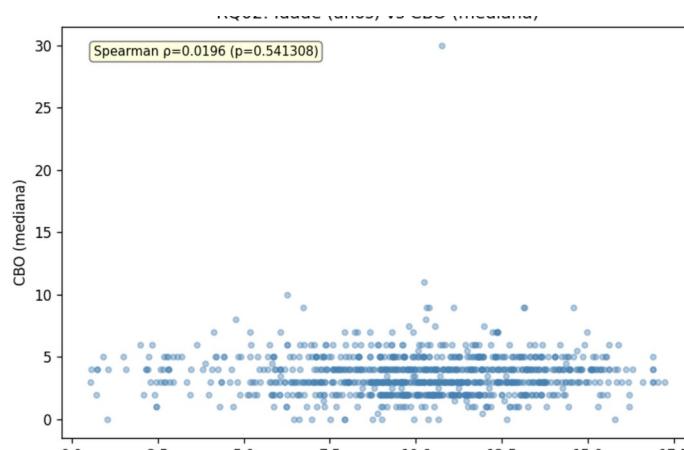


Figura 5 — Dispersão: idade $\times$ CBO. Spearman $\rho = 0,020$ ($p = 0,541$). Sem correlação.

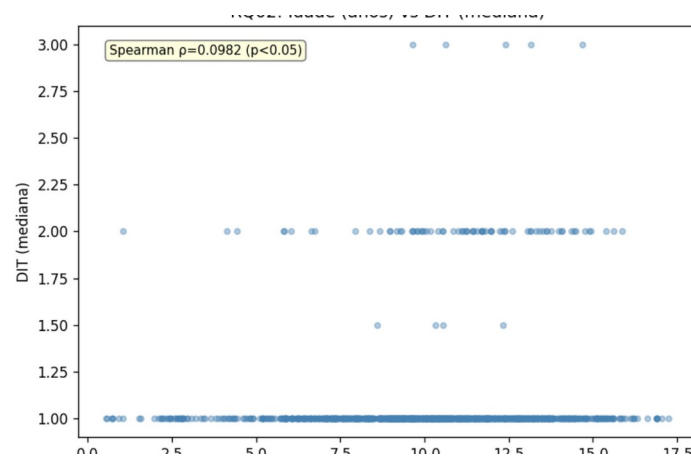


Figura 6 — Dispersão: idade $\times$ DIT. Spearman $\rho = 0,098$ ($p = 0,002$). Leve tendência positiva.

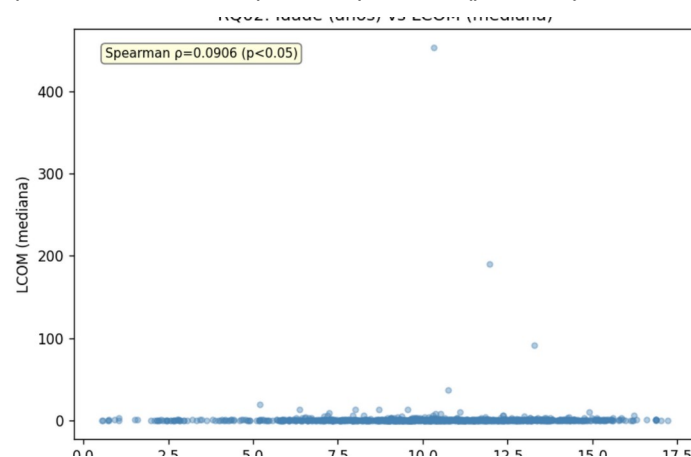


Figura 7 — Dispersão: idade $\times$ LCOM. Spearman $\rho = 0,091$ ($p = 0,005$). Mesmo padrão fraco.

Nota: Hipótese parcialmente confirmada. DIT e LCOM crescem levemente com a idade (dívida técnica acumulada), mas o efeito é fraco — a maturidade sozinha não prediz bem a qualidade. CBO não foi significativo.

6. RQ03 — Atividade vs. Qualidade

A atividade do repositório é quantificada pelo número total de releases publicadas. A hipótese era de que uma cadência alta de releases reflete manutenção ativa, o que favoreceria melhores valores de CBO e LCOM ao longo do tempo.

Medida	Valor
Mediana	11,00
Média	40,79
Desvio Padrão	89,28

Tabela 8 — Estatísticas descritivas: número de releases ($n = 978$).

A extrema assimetria (desvio padrão muito superior à média, que por sua vez é quase quatro vezes a mediana) reflete que poucos projetos concentram a maioria das releases — típico de grandes frameworks com ciclos de versionamento muito ativos. Os resultados vão na direção oposta à hipótese: projetos com mais releases apresentam mais acoplamento (CBO, $\rho = 0,287$) e menos coesão (LCOM, $\rho = 0,115$).

Métrica	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,287	< 0,001	Sim
DIT (mediana)	-0,053	0,099	Não
LCOM (mediana)	0,115	0,0003	Sim

Tabela 9 — Correlação de Spearman: número de releases $\times$ métricas de qualidade.

Para CBO, uma inclinação positiva moderada é visível na faixa de 0 a 300 releases. Para DIT, os pontos estão distribuídos sem padrão claro. Para LCOM, a tendência positiva existe mas é ofuscada pela concentração de valores próximos de zero. Uma interpretação plausível é que projetos com muitas releases são maiores e mais complexos — o crescimento funcional ao longo das versões introduz novas dependências sem refatoração estrutural. A hipótese é refutada.

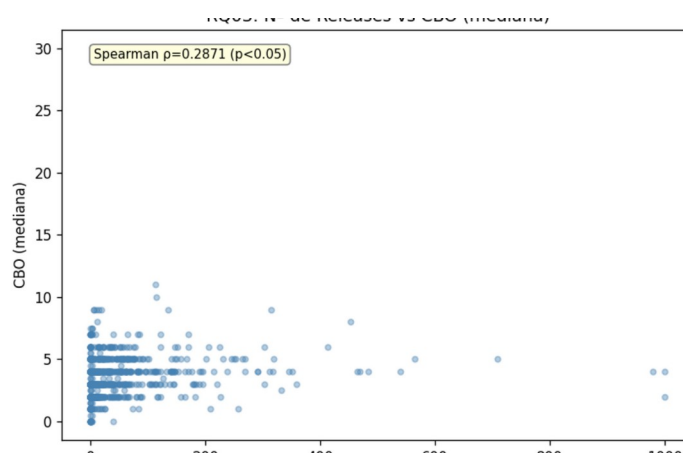


Figura 8 — Dispersão: releases $\times$ CBO. Spearman $\rho = 0,287$ ($p < 0,001$). Inclinação positiva visível.

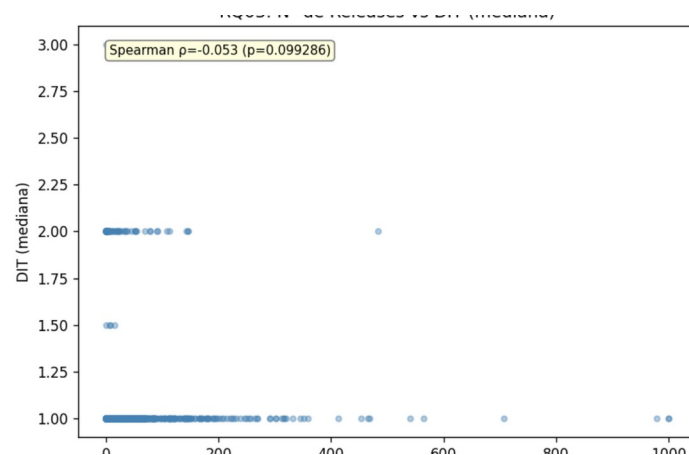


Figura 9 — Dispersão: releases $\times$ DIT. Spearman $\rho = -0,053$ ($p = 0,099$). Sem correlação.

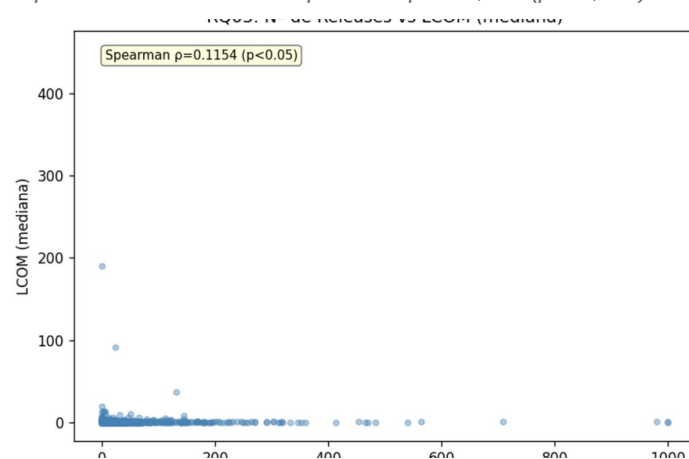


Figura 10 — Dispersão: releases $\times$ LCOM. Spearman $\rho = 0,115$ ($p < 0,001$). Tendência positiva.

Nota: Hipótese refutada — e invertida. Mais releases se associa a mais acoplamento e menos coesão. O crescimento funcional ao longo das versões introduz dependências sem refatoração estrutural correspondente.

7. RQ04 — Tamanho vs. Qualidade

O tamanho é avaliado por duas dimensões: a mediana de linhas de código por classe (LOC) e o total de linhas de comentário do repositório. A hipótese era de que projetos maiores apresentariam CBO e DIT mais elevados por conta de maior complexidade estrutural.

LOC — Linhas de Código (mediana por classe)

Medida	Valor
Mediana	17,00 linhas
Média	19,29 linhas
Desvio Padrão	13,23 linhas

Tabela 10 — Estatísticas descritivas: LOC mediana por classe ($n = 978$).

Métrica	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,429	< 0,001	Sim
DIT (mediana)	0,132	< 0,001	Sim
LCOM (mediana)	0,549	< 0,001	Sim

Tabela 11 — Correlação de Spearman: LOC mediana $\times$ métricas de qualidade.

Este foi o resultado mais forte do estudo. A correlação LOC vs. LCOM ($\rho = 0,549$) é moderada-forte: classes maiores tendem a ser substancialmente menos coesas, acumulando responsabilidades sem divisão clara de papéis. A correlação LOC vs. CBO ($\rho = 0,429$) é moderada: classes maiores também dependem de mais outras classes.

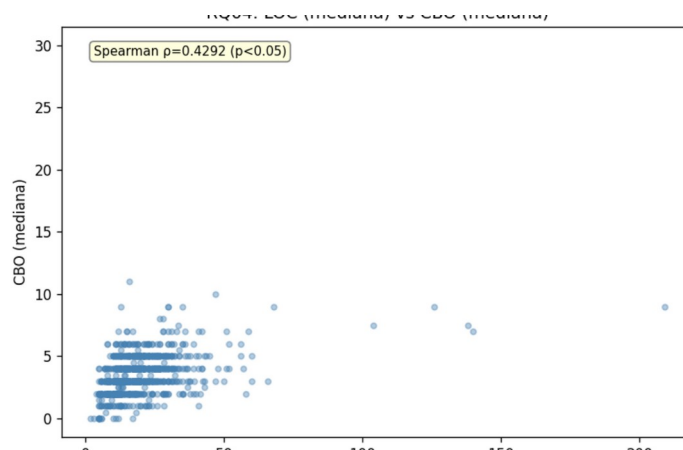


Figura 11 — Dispersão: LOC $\times$ CBO. Spearman $\rho = 0,429$ ($p < 0,001$). Correlação moderada.

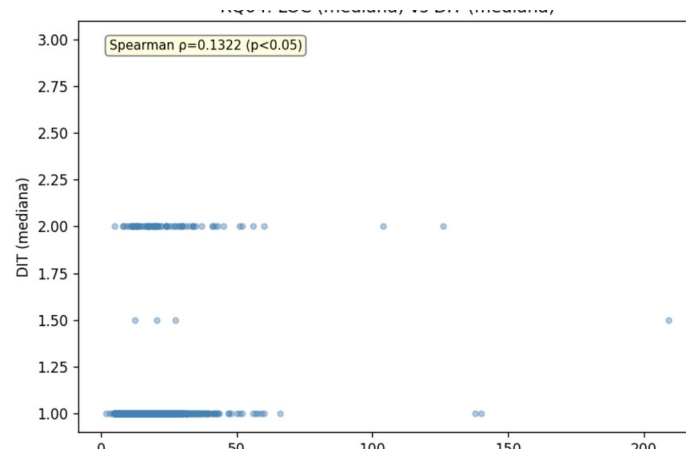


Figura 12 — Dispersão: LOC × DIT. Spearman $\rho = 0,132$ ($p < 0,001$). Correlação fraca.

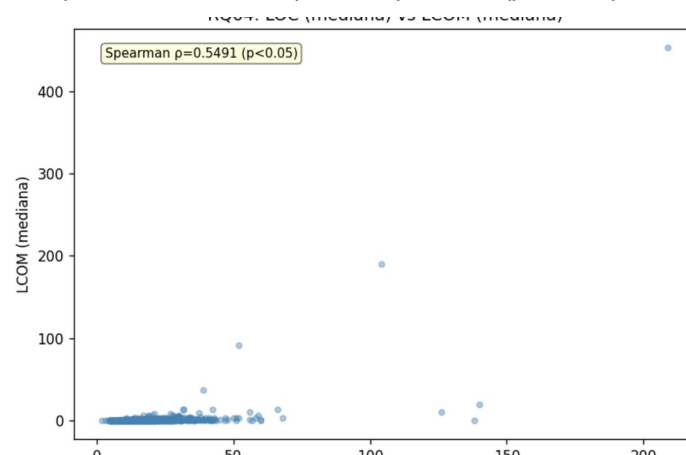


Figura 13 — Dispersão: LOC × LCOM. Spearman $\rho = 0,549$ ($p < 0,001$). Correlação mais forte do estudo.

Linhas de Comentário (total por repositório)

Medida	Valor
Mediana	4.757,50
Média	30.039,06
Desvio Padrão	75.268,15

Tabela 12 — Estatísticas descritivas: linhas de comentário totais ($n = 978$).

Métrica	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,305	< 0,001	Sim
DIT (mediana)	-0,048	0,136	Não
LCOM (mediana)	0,128	< 0,001	Sim

Tabela 13 — Correlação de Spearman: linhas de comentário × métricas de qualidade.

As linhas de comentário reforçam o mesmo padrão para CBO ($\rho = 0,305$), mas com intensidade menor. O alto desvio padrão ($dp = 75.268$) indica grande heterogeneidade: alguns projetos comentam extensivamente (possivelmente documentação de API), enquanto a maioria mantém poucas linhas.

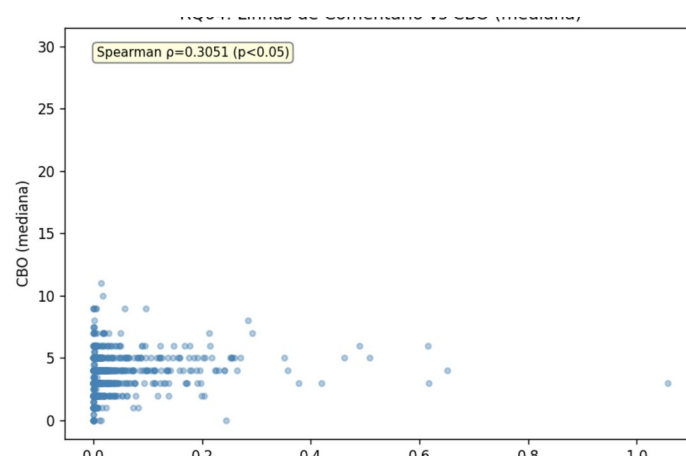


Figura 14 — Dispersão: comentários × CBO. Spearman $\rho = 0,305$ ($p < 0,001$).

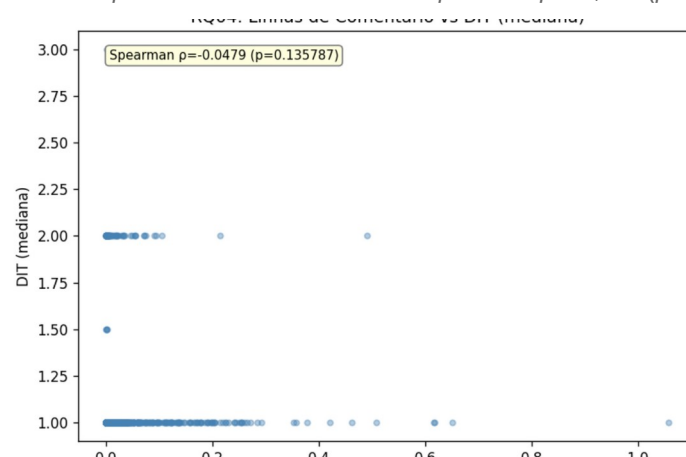


Figura 15 — Dispersão: comentários × DIT. Spearman $\rho = -0,048$ ($p = 0,136$). Sem correlação.

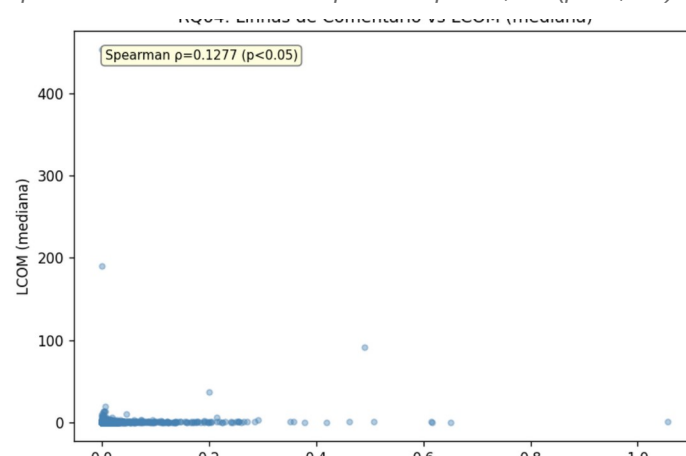


Figura 16 — Dispersão: comentários × LCOM. Spearman $\rho = 0,128$ ($p < 0,001$).

Nota: Hipótese confirmada. Tamanho é o preditor mais forte de degradação. LOC vs. LCOM ($\rho = 0,549$) é a correlação mais alta do estudo — classes maiores acumulam responsabilidades sem divisão clara de papéis.

8. Conclusão

Este estudo analisou empiricamente a qualidade interna de 978 repositórios Java populares no GitHub, correlacionando métricas de processo com métricas de qualidade calculadas pela ferramenta CK.

RQ	Hipótese	Resultado	Confirmada?
RQ01	Maior popularidade → menor CBO e LCOM	Correlações negligenciáveis ($\rho \approx 0$), não significativas	Não
RQ02	Maior maturidade → DIT e LCOM ligeiramente maiores	DIT ($\rho=0,098$) e LCOM ($\rho=0,091$) significativos; CBO não	Parcialmente
RQ03	Mais releases → melhor CBO e LCOM	CBO e LCOM aumentam com releases — sentido oposto à hipótese	Não
RQ04	Maior tamanho → CBO e LCOM mais altos	LOC vs LCOM ($\rho=0,549$) e LOC vs CBO ($\rho=0,429$) — confirmado	Sim

Tabela 14 — Síntese dos resultados por questão de pesquisa.

Os quatro achados principais do estudo são:

- **Popularidade não implica qualidade interna.** Estrelas medem visibilidade e adoção, não arquitetura. Projetos educacionais e frameworks amplamente adotados coexistem com perfis de qualidade estrutural muito distintos.
- **Tamanho é o preditor mais forte de degradação de qualidade.** A correlação LOC vs. LCOM ($\rho = 0,549$) é a mais alta do estudo, indicando que classes maiores acumulam responsabilidades de forma desproporcional.
- **Projetos mais ativos tendem a ser maiores e exibem mais acoplamento.** A correlação atividade vs. CBO ($\rho = 0,287$) reflete o crescimento acumulado de dependências ao longo das releases, sem refatoração estrutural correspondente.
- **Maturidade introduz leve acúmulo de dívida técnica.** As correlações fracas de DIT ($\rho = 0,098$) e LCOM ($\rho = 0,091$) com a idade sugerem que projetos mais antigos acumulam gradualmente hierarquias mais profundas e menor coesão.

Como limitações, destaca-se a exclusão de repositórios com sintaxe Java 14+ incompatível com o CK 0.7.0 — projetos mais recentes podem estar sub-representados. Adicionalmente, a análise representa uma fotografia temporal dos repositórios no momento da coleta.

Nota: 4 achados: (1) popularidade não implica qualidade; (2) tamanho é o maior preditor de degradação — LOC vs. LCOM $\rho=0,55$; (3) projetos mais ativos são maiores e acumulam mais acoplamento; (4) maturidade acumula dívida técnica leve em DIT e LCOM.